

Reviewer Pack — Real Radical Dimension Bounds AC^0 Circuit Size for PARITY

Entry 2c022a5d9d33 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 06:31:13 UTC

Real Radical Dimension Bounds AC^0 Circuit Size for PARITY

Entry ID: 2c022a5d9d33 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 06:31:13 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): AC^0 Circuit Size for PARITY

Statement:

For any AC^0 circuit C computing PARITY on n inputs, the dimension of the real radical of the ideal generated by C 's truth-table polynomial equations satisfies $\dim(\text{rad}(I_C)) \geq \log_2(\text{size}(C)) - O(\sqrt{n})$.

Rationale (proposer's reasoning):

Real radicals capture the algebraic structure of boolean functions; their dimension may encode circuit complexity via elimination theory. PARITY's inherent symmetry could create nontrivial constraints on radical varieties, linking algebraic geometry to circuit size.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 475c6e67e006221c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=241.8s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    size_C = random.randint(2, 100) # Randomly generate a size for AC0 circuit C

    # Generate truth-table polynomial equations for PARITY on n inputs
    truth_table = [[i & (1 << j) % 2 for j in range(n)] for i in range(1 << n)]

    # Construct the ideal I_C from truth-table equations
    I_C = []
    for row in truth_table:
        poly = sum(row[j] * x**j for j in range(n)) - sum(row[j] for j in range(n))
        I_C.append(poly)

    # Compute the real radical's dimension via Gröbner bases (simplified version)
    def gcd(a, b):
        while b:
            a, b = b, a % b
        return a

    def lcm(a, b):
        return abs(a * b) // gcd(a, b)

    def simplify(poly):
        return [coeff for coeff in poly if coeff != 0]

    def multiply(p1, p2):
        result = [0] * (len(p1) + len(p2))
        for i in range(len(p1)):
            for j in range(len(p2)):
                result[i + j] += p1[i] * p2[j]
        return simplify(result)

    def divide(poly, divisor):
        quotient = []
        remainder = poly[:]
        while remainder and remainder[0] != 0:
            degree = len(remainder) - 1
            coeff = remainder[degree] / divisor[-1]
            quotient.append(coeff)
            for i in range(degree + 1):
```

```

        remainder[i] -= coeff * divisor[i]
        remainder = simplify(remainder)
    return quotient, remainder

def gcd_poly(p1, p2):
    if not p2:
        return p1
    return gcd_poly(p2, [p1[i] % p2[i] for i in range(len(p1))])

def lcm_poly(p1, p2):
    return divide(gcd_poly(p1, p2), 1)[0]

# Simplify the ideal I_C
I_C = simplify(I_C)

# Compute the dimension of the real radical (simplified version)
dim_rad_I_C = len(I_C) - len(set(I_C))

# Verify the conjecture
if dim_rad_I_C >= math.log2(size_C) - 7:
    conjecture_holds = True
    counterexample = ""
else:
    conjecture_holds = False
    counterexample = "dim(rad(I_C)) < log2(size(C)) - 7"

return {
    "metric_name": "dimension of real radical",
    "metric_value": dim_rad_I_C,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2*i + 1 for i in range(5, 30)] # Default list of 30 primes

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(res["metric_value"] for res in results) / len(results)
    std_metric_value = math.sqrt(sum((res["metric_value"] - mean_metric_value)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.4f} std={std_metric_value:.4f} support={support_fraction:.4f}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next((seed for seed, res in zip(seeds, results) if not res["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample=\"{res['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_support_fraction support_fraction={support_fraction:.4f}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results; support fraction cannot be evaluated | next: Run test with extended timeout and verify implementation correctness

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	136172
2	propose	ollama_local	qwen3:8b	0	0	202052
3	preregistration	ollama_remote	qwen3:8b	0	0	31976
4	novelty	ollama_remote	qwen3:8b	0	0	29256
5	novelty	ollama_remote	qwen3:8b	0	0	19731
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15700
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7642
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7644
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11384
10	judge	ollama_remote	qwen3:8b	0	0	117360

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 578918 ms total latency. Provider mix: {'ollama_remote': 9, 'ollama_local': 1}

(full prompt+response transcripts available in `research/audit/2c022a5d9d33.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2c022a5d9d33.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2c022a5d9d33.tar.gz` (if generated)